



**Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA**

YouTube Video Link: <https://youtu.be/X1zAjDLTaSo>

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table><tr><th>Student 1</th><th>Student 2</th></tr><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></table> Name 1: Lam Yoke Yu Metric No: A23CS0233 Name 2: Tan Yi Ya Metric No: A23CS0187	Student 1	Student 2	<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>
Student 1	Student 2				
<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>				
SUBMISSION DATE:	22/05/2024				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE 1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE 2	BX = 0009h var2 = 0009h	Move the value var2 (9d) into register BX
LINE 3	AX = 0009h BX = 0001h	Exchange the values of AX and BX
LINE 4	AX = 0009h var1 = 0009h	Move the value of register AX into variable var1
LINE 5	BX = 0001h var2 = 0001h	Move the value of register BX into variable var2

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 006F0001 EBX = 0051E000 ECX = 00D210AA EDX = 00D210AA				
ESI = 00D210AA EDI = 00D210AA EIP = 00D23666 ESP = 006FFB10				
EBP = 006FFB1C EFL = 00000246				

LINE2:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 006F0001 EBX = 00510009 ECX = 00D210AA EDX = 00D210AA ESI = 00D210AA EDI = 00D210AA EIP = 00D2366D ESP = 006FFB10 EBP = 006FFB1C EFL = 00000246				

LINE3:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 006F0009 EBX = 00510001 ECX = 00D210AA EDX = 00D210AA ESI = 00D210AA EDI = 00D210AA EIP = 00D2366F ESP = 006FFB10 EBP = 006FFB1C EFL = 00000246				

LINE4:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 006F0009 EBX = 00510001 ECX = 00D210AA EDX = 00D210AA ESI = 00D210AA EDI = 00D210AA EIP = 00D23675 ESP = 006FFB10 EBP = 006FFB1C EFL = 00000246				

LINE5:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 006F0009 EBX = 00510001 ECX = 00D210AA EDX = 00D210AA ESI = 00D210AA EDI = 00D210AA EIP = 00D2367C ESP = 006FFB10 EBP = 006FFB1C EFL = 00000246				

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval       ; LINE3
    sub ebx,Zval       ; LINE4
    add eax,ebx        ; LINE5
    inc eax            ; LINE6
    mov Rval,eax       ; LINE7
    exit
main endp
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

b) Table 1

LINE 1	EAX = 0000001Ah Xvar = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE 2	EAX = FFFFFFFE6h	Convert the value of 26d to its 2's complement (-26d)
LINE 3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE 4	EBX = FFFFFFF6h Zval = 00000028h	Subtract the value of Zval (40d) from register EBX (30d) $EBX = 30d - 40d = -10d$
LINE 5	EAX = FFFFFFFDCh EBX = FFFFFFF6h	Add the value of EBX (-10d) to EAX (-26d) $EAX = -26d + (-10d) = -36d$
LINE 6	EAX = FFFFFFFDDh	Increase the value of EAX (-36d) by 1 and become -35d
LINE 7	EAX = FFFFFFFDDh Rval = FFFFFFFDDh	Move the value of EAX (-35d) into variable Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 0000001A EBX = 011C3000 ECX = 000A1005 EDX = 000A1005				
ESI = 000A1005 EDI = 000A1005 EIP = 000A1015 ESP = 012FFC54				
EBP = 012FFC60 EFL = 00000246				

LINE2:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = FFFFFFFE6	EBX = 011C3000	ECX = 000A1005	EDX = 000A1005	
ESI = 000A1005	EDI = 000A1005	EIP = 000A1017	ESP = 012FFC54	
EBP = 012FFC60	EFL = 00000293			

0x000A4008 = 0000001E

LINE3:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = FFFFFFFE6	EBX = 0000001E	ECX = 000A1005	EDX = 000A1005	
ESI = 000A1005	EDI = 000A1005	EIP = 000A101D	ESP = 012FFC54	
EBP = 012FFC60	EFL = 00000293			

0x000A400C = 00000028

LINE4:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = FFFFFFFE6	EBX = FFFFFFFF6	ECX = 000A1005	EDX = 000A1005	
ESI = 000A1005	EDI = 000A1005	EIP = 000A1023	ESP = 012FFC54	
EBP = 012FFC60	EFL = 00000287			

LINE5:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = FFFFFFFDC	EBX = FFFFFFFF6	ECX = 000A1005	EDX = 000A1005	
ESI = 000A1005	EDI = 000A1005	EIP = 000A1025	ESP = 012FFC54	
EBP = 012FFC60	EFL = 00000283			

LINE6:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = FFFFFFFDD	EBX = FFFFFFFF6	ECX = 000A1005	EDX = 000A1005	
ESI = 000A1005	EDI = 000A1005	EIP = 000A1026	ESP = 012FFC54	
EBP = 012FFC60	EFL = 00000287			

0x000A4000 = 00000000

LINE7:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = FFFFFFFDD	EBX = FFFFFFFF6	ECX = 000A1005	EDX = 000A1005	
ESI = 000A1005	EDI = 000A1005	EIP = 000A102B	ESP = 012FFC54	
EBP = 012FFC60	EFL = 00000287			

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

a) Fill (Write) in the contents for the related register in each line:

Table 3

LINE 1	EAX = 00000005h var1 = 00000005h	Move the value of val1(5d) into register EAX
LINE 2	EAX = 00000032h var2 = 0000000Ah	Multiply the value of EAX (5d) by var2 (10d)
LINE 3	EAX = 00000046h var3 = 00000014h	Add the value of var3 (20d) to EAX (50d)
LINE 4	EAX = 00000045h var4 = 00000000h	Decrement of the value of EAX (70d) by 1 var4 is not assigned with any value

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 00000005 EBX = 00DE4000 ECX = 00F81005 EDX = 00F81005 ESI = 00F81005 EDI = 00F81005 EIP = 00F81015 ESP = 00EFF78C EBP = 00EFF798 EFL = 00000246				
0x00F84004 = 0000000A				

LINE2:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 00000032 EBX = 00DE4000 ECX = 00F81005 EDX = 00000000				
ESI = 00F81005 EDI = 00F81005 EIP = 00F8101B ESP = 00EFF78C				
EBP = 00EFF798 EFL = 00000202				

0x00F84008 = 00000014

LINE3:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 00000046 EBX = 00DE4000 ECX = 00F81005 EDX = 00000000				
ESI = 00F81005 EDI = 00F81005 EIP = 00F81021 ESP = 00EFF78C				
EBP = 00EFF798 EFL = 00000202				

LINE4:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 00000045 EBX = 00DE4000 ECX = 00F81005 EDX = 00000000				
ESI = 00F81005 EDI = 00F81005 EIP = 00F81022 ESP = 00EFF78C				
EBP = 00EFF798 EFL = 00000202				

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE 1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE 2	BX = 0005h	Move the value of 5d into register BX
LINE 3	AX = 00C8h BX = 0005h	Multiply the value of AX (40d) by value of BX (5d)
LINE 4	BX = 000Ah var2 = 000Ah	Move the value of var2 (10d) into register BX
LINE 5	BX = 0007h	Subtract the value of BX (10d) by 3d
LINE 6	AX = 001Ch BX = 0007h DX = 0004h	Divide the value of AX (200d) by BX (7d) The quotient (28d) is stored in register AX The remainder (4d) is stored in register DX
LINE 7	AX = 001Ch var4 = 001Ch	Move the value of AX (28d) to var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 00900028	EBX = 00BB8000	ECX = 009F1005	EDX = 009F1005	
ESI = 009F1005	EDI = 009F1005	EIP = 009F1016	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000246			

LINE2:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 00900028	EBX = 00BB0005	ECX = 009F1005	EDX = 009F1005	
ESI = 009F1005	EDI = 009F1005	EIP = 009F101A	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000246			

LINE3:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 009000C8	EBX = 00BB0005	ECX = 009F1005	EDX = 009F0000	
ESI = 009F1005	EDI = 009F1005	EIP = 009F101D	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000202			

LINE4:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 009000C8	EBX = 00BB000A	ECX = 009F1005	EDX = 009F0000	
ESI = 009F1005	EDI = 009F1005	EIP = 009F1024	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000202			

LINE5:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 009000C8	EBX = 00BB0007	ECX = 009F1005	EDX = 009F0000	
ESI = 009F1005	EDI = 009F1005	EIP = 009F1028	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000202			

LINE6:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 0090001C	EBX = 00BB0007	ECX = 009F1005	EDX = 009F0004	
ESI = 009F1005	EDI = 009F1005	EIP = 009F102B	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000202			

LINE7:

Registers	Autos	Watch 1	Memory 1	Disassembly
EAX = 0090001C	EBX = 00BB0007	ECX = 009F1005	EDX = 009F0004	
ESI = 009F1005	EDI = 009F1005	EIP = 009F1031	ESP = 0090FF28	
EBP = 0090FF34	EFL = 00000202			

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
- b. The divisor is specified as the operand of the DIV instruction.
- c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
- d. The remainder is stored in DX.
- e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
- f. If the divisor is 0, a division error occurs.
- g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

$4096 * 37 = 151552$

$1000h * 25h = 25000h$ (DX:AX)

; Code Snippet 1 (MUL CX)

```
MOV DX, 0           ; Clear DX
MOV AX, 1000h        ; Load 1000h into AX
MOV CX, 25h          ; Load 25h into CX
MUL CX               ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

- a) Screenshot of full program (.asm) :

```

main.asm  + X
lab 2.5 (Global Scope)
1  TITLE lab 2.5
2  ; Author: Lam Yoke Yu & Tan Yi Ya
3  ; Date: 17 May 2024
4  include Irvine32.inc
5
6  .code
7  main proc
8      MOV DX, 0          ; Clear DX
9      MOV AX, 1000h      ; Load 1000h into AX
10     MOV CX, 25h        ; Load 25h into CX
11     MUL CX             ; Multiply AX by CX, storing the result in DX : AX
12     call dumpRegs
13     exit
14 main ENDP
15 END main

```

b) Paste here the screenshot of the final registers' content (DumpReg):

```

Microsoft Visual Studio Debu  X  +  v  -  □  X

EAX=004F5000  EBX=002F6000  ECX=005B0025  EDX=005B0002
ESI=005B10AA  EDI=005B10AA  EBP=004FF88C  ESP=004FF880
EIP=005B3674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1

```

Q6. Given the following instructions as is Code Snippet 2.

- Write a full program to execute the Code Snippet 2.
- What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

$$2051 / 16 = 128 \text{ r } 3$$

$$803\text{h} / 10\text{h} = 80\text{h} \text{ r } 3\text{h} \text{ (AL r AH)}$$

```

; Code Snippet 2 (DIV BL)
MOV DX, 0          ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h       ; Load the dividend (8003h) into AX
MOV BL, 10h        ; Load the divisor (10h) into BL
DIV BL             ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder

```

Answer Q6

- Screenshot of full program (.asm) :

```
main.asm*  X
lab 2.6 (Global :
1  TITLE lab 2.6
2  ; Author: Lam Yoke Yu & Tan Yi Ya
3  ; Date: 17 May 2024
4  include Irvine32.inc
5
6  .code
7  main proc
8      MOV DX, 0
9      MOV AX, 803h
10     MOV BL, 10h
11     DIV BL
12     call dumpRegs
13     exit
14 main ENDP
15 END main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
Microsoft Visual Studio Debug Console
EAX=012F0380  EBX=0117D010  ECX=00C410AA  EDX=00C40000
ESI=00C410AA  EDI=00C410AA  EBP=012FF9B0  ESP=012FF9A4
EIP=00C43671  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1
```